Installation and Setup Guide

This guide provides step-by-step instructions for installing and configuring the Azure DevOps Permission Matrix Toolkit.

System Requirements

Minimum Requirements

- **Operating System**: Windows 10/11, Windows Server 2016+, macOS 10.14+, or Linux
- **PowerShell**: Version 5.1 or later (Windows PowerShell) or PowerShell 7.0+ (PowerShell Core)
- **Memory**: 4 GB RAM minimum, 8 GB recommended
- **Disk Space**: 100 MB for toolkit files, additional space for reports
- **Network**: Internet connectivity to access Azure DevOps Services

Azure DevOps Requirements

- Azure DevOps Services organization (cloud)
- User account with appropriate permissions
- Personal Access Token with required scopes

Installation Steps

Step 1: Download and Extract Toolkit

1. Extract the toolkit package to your desired location:
   `C:\Tools\AzureDevOpsToolkit\`

2. Verify the folder structure:
   ```
   AzureDevOpsToolkit/
   ├── scripts/
   ├── templates/
   ├── docs/
   ├── checklists/
   └── build/
   ```

Step 2: PowerShell Environment Setup

Windows PowerShell (5.1)

1. Open PowerShell as Administrator
2. Check execution policy:
   ```
   powershell
   Get-ExecutionPolicy
   ```
3. If restricted, set execution policy:
   ```
   powershell
   Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
   ```

PowerShell Core (7.0+)

1. Install PowerShell 7+ if not already installed:
   - Download from: https://github.com/PowerShell/PowerShell/releases
   - Or use package manager (Windows): `winget install Microsoft.PowerShell`

2. Verify installation:
   `powershell`
   `$PSVersionTable.PSVersion`

Step 3: Install Required Modules

ImportExcel Module (Optional but Recommended)

For Excel report generation:

```
# Install for current user
Install-Module -Name ImportExcel -Scope CurrentUser -Force

# Verify installation
Get-Module -ListAvailable ImportExcel
```

Alternative Installation Methods

If you encounter issues with Install-Module:

1. **Manual Download**:
   - Download from PowerShell Gallery: https://www.powershellgallery.com/packages/ImportExcel
   - Extract to PowerShell modules directory

2. **Offline Installation**:
   ```powershell
   # On internet-connected machine
   Save-Module -Name ImportExcel -Path “C:\Temp\Modules”

# Copy to target machine and import
Import-Module “C:\Temp\Modules\ImportExcel”
```

Step 4: Azure DevOps Personal Access Token Setup

Creating a Personal Access Token

1. **Sign in to Azure DevOps**:
   - Navigate to https://dev.azure.com/yourorganization
   - Sign in with your credentials

2. **Access User Settings**:
   - Click your profile picture (top right)
   - Select “Personal access tokens”

3. **Create New Token**:
   - Click “New Token”
   - Provide a descriptive name: “Permission Audit Toolkit”
   - Set expiration (90 days recommended for security)
   - Select organization scope

4. **Configure Required Scopes**:
   Select the following scopes:
   - **Project and Team** (Read)
   - **Identity** (Read)
   - **Security** (Manage)
   - **Code** (Read)
   - **Build** (Read)
   - **Release** (Read)
   - **Service Connections** (Read)
   - **Agent Pools** (Read)
   - **Variable Groups** (Read)

5. **Save Token Securely**:
   - Copy the token immediately (it won't be shown again)
   - Store in a secure location (password manager recommended)

Token Security Best Practices

```powershell
# Store token as secure string (recommended)
$secureToken = Read-Host "Enter PAT" -AsSecureString
$pat = [System.Runtime.InteropServices.Marshal]::PtrToStringAuto([Sys-
tem.Runtime.InteropServices.Marshal]::SecureStringToBSTR($secureToken))

# Or use environment variable
$env:AZURE_DEVOPS_PAT = "your_token_here"
```

Step 5: Verify Installation

Test Basic Functionality

1. **Navigate to scripts directory**:
   `powershell`
   `cd "C:\Tools\AzureDevOpsToolkit\scripts"`

2. **Test organization connection**:
   `powershell`
   `.\Get-AzureDevOpsOrgPermissions.ps1 -OrganizationUrl "https://dev.azure.com/yourorg" -PersonalAccessToken "your_token" -OutputPath "test_output.csv"`

3. **Verify output file creation**:
   `powershell`
   `Test-Path "test_output.csv"`
   `Get-Content "test_output.csv" | Select-Object -First 5`

Test Excel Generation (if ImportExcel installed)

```powershell
.\Run-AzureDevOpsPermissionAudit.ps1 -OrganizationUrl "https://dev.azure.com/yourorg" -
PersonalAccessToken "your_token" -GenerateExcelReport
```

Configuration Options

Script Parameters Configuration

Create a configuration file for repeated use:

```
# config.ps1
$AuditConfig = @{
    OrganizationUrl = "https://dev.azure.com/yourorg"
    PersonalAccessToken = $env:AZURE_DEVOPS_PAT
    OutputDirectory = "C:\AuditReports"
    GenerateExcelReport = $true
}

# Usage
.\Run-AzureDevOpsPermissionAudit.ps1 @AuditConfig
```

Scheduled Execution Setup

Windows Task Scheduler

1. **Create PowerShell script wrapper**:
   `powershell`
   `# audit_wrapper.ps1`
   `Set-Location "C:\Tools\AzureDevOpsToolkit\scripts"`
   `$pat = Get-Content "C:\secure\pat.txt" # Store PAT securely`
   `.\Run-AzureDevOpsPermissionAudit.ps1 -OrganizationUrl "https://dev.azure.com/yourorg" -PersonalAccessToken $pat -OutputDirectory "C:\AuditReports\$(Get-Date -Format 'yyyy-MM')"`

2. **Create scheduled task**:
   - Open Task Scheduler
   - Create Basic Task
   - Set trigger (e.g., monthly)
   - Action: Start a program
   - Program: `powershell.exe`
   - Arguments: `-File "C:\Tools\AzureDevOpsToolkit\audit_wrapper.ps1"`

Linux/macOS Cron Job

```
# Add to crontab (monthly execution)
0 0 1 * * /usr/bin/pwsh -File /opt/azure-devops-toolkit/scripts/audit_wrapper.ps1
```

Network and Firewall Configuration

Required Network Access

Ensure the following endpoints are accessible:

- `https://dev.azure.com` (Azure DevOps Services)
- `https://app.vssps.visualstudio.com` (Visual Studio Services)
- `https://management.azure.com` (Azure Resource Manager - if using Azure integrations)

Proxy Configuration

If using a corporate proxy:

```powershell
# Set proxy for PowerShell session
$proxy = New-Object System.Net.WebProxy("http://proxy.company.com:8080")
$proxy.Credentials = [System.Net.CredentialCache]::DefaultCredentials
[System.Net.WebRequest]::DefaultWebProxy = $proxy
```

Troubleshooting Installation Issues

Common PowerShell Issues

1. **Execution Policy Error**:
   `Error: Execution of scripts is disabled on this system`
   Solution: Set execution policy as shown in Step 2

2. **Module Installation Fails**:
   `Error: Unable to install module 'ImportExcel'`
   Solutions:
   - Run PowerShell as Administrator
   - Use `-Force` parameter
   - Try manual installation method

3. **TLS/SSL Errors**:
   `Error: The request was aborted: Could not create SSL/TLS secure channel`
   Solution:
   `powershell`
   `[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12`

Azure DevOps Connection Issues

1. **Authentication Failures**:
   - Verify PAT is not expired
   - Check PAT scopes include required permissions
   - Ensure organization URL is correct format

2. **Permission Denied Errors**:
   - Verify user has access to organization/projects
   - Check PAT scopes are sufficient
   - Confirm user is not restricted by conditional access policies

Performance Optimization

For large organizations:

1. **Increase timeout values**:
   `powershell`
   `# Add to scripts if needed`
   `$PSDefaultParameterValues['Invoke-RestMethod:TimeoutSec'] = 300`

2. **Run analysis in batches**:
   `powershell`
   `# Analyze specific projects instead of all`
   `.\Run-AzureDevOpsPermissionAudit.ps1 -ProjectName "CriticalProject1"`

Next Steps

After successful installation:

1. Review `docs/BEST_PRACTICES.md` for operational guidance
2. Examine `checklists/checklist.md` for Azure DevOps setup recommendations
3. Run initial audit to establish baseline
4. Set up regular audit schedule
5. Configure report distribution to stakeholders

Support

For additional support:
- Check `docs/TROUBLESHOOT.md` for detailed troubleshooting
- Review PowerShell execution logs
- Verify Azure DevOps service status at https://status.dev.azure.com/

Installation complete! You're ready to begin auditing Azure DevOps permissions.